BACHELOR OF TECHNOLOGY / PROJECT REPORT

EcoSort AI

AI-Powered Waste Classification

A full project report on a transfer-learning image classifier and mobile-friendly web application for waste identification.

SUBMITTED BY	SUBMITTED TO
Arshpreet Singh Sarabjit Singh Pooja	Project Mentor: Dr. Brijesh Bakariya Project Co-Mentor: Er. Satyam Sharma Department of Computer Science and Engineering IK Gujral Punjab Technical University, Hoshiarpur Campus
Semester: 7th Roll numbers:2323174, 2323218,2323200	Academic session: 2027 Submission date:

Submitted in partial fulfilment of the requirements for the award of the degree of [Degree / Programme].

Declaration

We, the undersigned project team, declare that this report describes the EcoSort AI project work carried out by our team. Project-specific implementation details and reported metrics are drawn from the project repository and its saved model metadata. Any external source used for technical background is listed in the references. We understand that this declaration should be retained only after review and approval by every listed student.

Student signatures

Name	Signature	Date
Arshpreet Singh	_____	_____
Sarabjit Singh	_____	_____
Pooja	_____	_____

Institutional details to complete

Department: Computer Science and Engineering
Institution: IK Gujral Punjab Technical University, Hoshiarpur Campus
Project title as approved by department: Garbage Classification using Deep Learning
Academic session: 2027

Certificate

This certificate page is prepared for review and signature by the project mentors and department. It does not imply approval until the authorized signatories have completed it.

CERTIFICATION

This is to certify that the project entitled **EcoSort AI: AI-Powered Waste Classification** has been carried out by Arshpreet Singh, Sarbjit Singh, and Pooja in the Department of Computer Science and Engineering, IK Gujral Punjab Technical University, Hoshiarpur Campus. The project was completed under the mentorship of **Dr. Brijesh Bakariya, Project Mentor and Head of Department**, with **Er. Satyam Sharma, Project Co-Mentor**, in partial fulfilment of the requirements for [Degree / Programme] during [Academic Session]. This certification is subject to institutional review and approval.

Project Mentor / HOD	Project Co-Mentor	External Examiner
Dr. Brijesh Bakariya	Er. Satyam Sharma	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Institution seal / remarks

_____

_____

Abstract

EcoSort AI is a deep-learning project that classifies a waste-item image into one of twelve supported dataset categories and presents the result through a responsive web application. The system combines a MobileNetV2 transfer-learning model with a Flask server and a browser interface that supports both photo upload and camera capture. In camera mode, the user sees a live preview, captures a still, reviews it, and explicitly starts prediction; the app does not continuously classify the video feed.

The repository dataset contains 15,515 images distributed among battery, biological, brown glass, cardboard, clothes, green glass, metal, paper, plastic, shoes, trash, and white glass. Images are loaded at 224 by 224 pixels and split into training and validation sets using an 80/20 configuration and seed 42. The model starts from ImageNet-pretrained MobileNetV2 weights, uses a task-specific twelve-class softmax head, class-weighted training, augmentation, early stopping, and a fine-tuning stage for the final thirty backbone layers.

The saved metadata reports 94.68% validation accuracy and 0.172 validation loss. These values describe the selected model on the project's validation split; they are not an independent test result or a guarantee for new user photographs. The project outcome is an end-to-end prototype that connects dataset preparation, model training, server-side inference, image upload, camera capture, and a mobile-installable web experience. Further work should establish data provenance, evaluate a held-out test set, report per-class metrics, and improve low-confidence and multi-object handling.

Keywords: waste classification, computer vision, transfer learning, MobileNetV2, TensorFlow, Flask, camera capture, Progressive Web App

Acknowledgements

The project team expresses its sincere gratitude to **Dr. Brijesh Bakariya**, Project Mentor and Head of the Department of Computer Science and Engineering, IK Gujral Punjab Technical University, Hoshiarpur Campus, for his academic guidance and support. The team also thanks **Er. Satyam Sharma**, Project Co-Mentor, for his guidance and feedback during the project work.

The team acknowledges the Department of Computer Science and Engineering and IK Gujral Punjab Technical University, Hoshiarpur Campus, for the academic environment in which this project was developed. We also acknowledge the authors and maintainers of the open-source frameworks listed in the references. Dataset provenance and licensing details were not available in the supplied repository and should be confirmed separately.

Project team

Arshpreet Singh
Sarabjit Singh
Pooja

Table of Contents

Chapter page references are generated during PDF build.

01 | Introduction 8

02 | Objectives and Scope 9

03 | Technical Background and Related Work 10

04 | System Overview 11

05 | Dataset Description 12

06 | Tools and Technologies 13

07 | Data Preprocessing 14

08 | Methodology 15

09 | Model Architecture and Training 16

10 | Web Application Implementation 17

11 | System Architecture 18

12 | User Interface Screenshots 19

13 | Results and Discussion 20

14 | Verification and Test Plan 21

15 | Deployment and Operation 22

16 | Privacy, Security, and Operational Risks 23

17 | Advantages and Limitations 24

18 | Future Scope 25

19 | Software Stack at a Glance 26

20 | Python Runtime and Dependency Management 27

21 | TensorFlow and Keras 28

22 | MobileNetV2 Architecture 29

23 | Classifier Head and Transfer-Learning Stages 30

24 | Augmentation and Class Imbalance 31

25 | Training Controls and Reproducibility 32

26 | NumPy, Pillow, and Image Preparation 33

27 | Flask Backend and Prediction API 34

28 | Browser Interface: HTML, CSS, and JavaScript 35

29 | Progressive Web App and Camera APIs 36

30 | Interface Screenshot: Upload Workflow 37

31 | Interface Screenshot: Camera Capture 38

32 | Dataset Sample Gallery 39

33 | Class Distribution and Results Interpretation 40

34 | User Guide: Photo Upload 41

35 | User Guide: Camera and Troubleshooting 42

36 | Deployment Environment and Compatibility 43

37 | Integration, Verification, and Traceability 44

38 | Conclusion 45

39 | Appendices 47

01 | Introduction

1.1 Background

Waste sorting is a repeated everyday decision. Packaging, paper products, glass, clothing, batteries, and organic material require different collection or treatment pathways. People may be uncertain about an item's dominant material, especially when the object is damaged, mixed-material, or visually similar to another category. This project investigates a focused technical question: can a supervised image classifier provide a useful first-pass category suggestion through a simple browser-based photo workflow?

EcoSort AI connects an image dataset and a transfer-learning model to an application interface. A user can upload a photo or use a camera to capture a still image. The server preprocesses the image, runs the selected Keras model, and returns ranked category scores. The system is intended as a learning prototype and decision aid, not as a municipal recycling authority.

1.2 Problem statement

A user-facing waste classifier must solve more than model inference. It must make the input workflow understandable, handle supported image formats, preprocess an image consistently with training, load the intended class ordering, return interpretable results, and communicate the limits of a single-label prediction. Camera use also requires permission and a secure browser context. The project therefore treats model training and the surrounding web workflow as one integrated system.

1.3 Report organization

The chapters describe the project objectives and boundaries; technical background; existing and proposed approaches; the dataset; software tools; image preparation; methodology; model architecture and training; application implementation; system architecture; interface screenshots; evaluation; verification planning; deployment; risks; future scope; and conclusion. The final sections provide references and practical appendices.

02 | Objectives and Scope

2.1 Project objectives

- Prepare a supervised image-classification pipeline for twelve waste categories in the supplied dataset.

- Use transfer learning with MobileNetV2 and train a project-specific twelve-class classifier.

- Expose model inference through a browser app with image upload and ranked prediction scores.

- Provide a camera workflow with visible preview, still-image capture, review, and user-triggered prediction.

- Support a responsive mobile experience and package the web shell as an installable PWA where browser support permits.

- Record model results, technical constraints, and future evaluation needs without overstating validation performance.

2.2 Scope

The implemented task is image-level, single-label classification across the twelve configured classes. The output is a ranking of likely known classes, with the top five returned by the server. The application does not perform object detection, segmentation, multi-item localization, material-composition analysis, contamination grading, barcode or recycling-symbol reading, or jurisdiction-specific disposal lookup.

2.3 Intended use and exclusions

The prototype is most appropriate for a clear image with one visible item. Users should treat its output as a suggestion and check official local guidance, particularly for batteries and other hazardous or specially collected materials. The project is not a safety-critical tool, a waste-collection management system, or a substitute for trained inspection.

03 | Technical Background and Related Work

3.1 Image classification

Image classification maps an input image to one label from a predefined set. A convolutional neural network learns visual representations through layers that respond to patterns such as edges, textures, shapes, and object parts. In a supervised setting, the training examples have labels and the model parameters are updated to reduce a classification loss. The final softmax layer expresses a normalized distribution over the configured categories. Those scores are model outputs; without calibration analysis, they should not be interpreted as guaranteed probabilities of correctness.

3.2 Transfer learning

Training a deep network from random initialization requires substantial labeled data and compute. Transfer learning starts from a model whose visual features were learned on a large source dataset, then adapts some or all of the representation to a target task. In this project, MobileNetV2 provides the feature-extraction backbone and a new dense classifier is trained for the repository's waste labels. A staged process first trains the new head while the backbone is frozen and then fine-tunes selected backbone layers at a lower learning rate.

3.3 MobileNetV2

MobileNetV2 was designed as an efficient convolutional architecture using inverted residual blocks and linear bottlenecks. Its design targets favorable trade-offs among model capacity, computation, and memory on constrained devices [1]. In EcoSort AI it is used as a pretrained feature extractor on the server, not as an on-device mobile model. The fact that the architecture is mobile-oriented does not mean this application performs offline inference on a phone.

3.4 Application-layer concepts

Flask provides URL routing and request handling for the Python web service [3]. TensorFlow/Keras provides model construction, training, serialization, and inference. The browser handles file selection, camera permission, still capture, and result presentation. A PWA manifest and service worker support installation and caching of the web shell, but the saved neural-network model remains on the server and predictions require connectivity.

3.5 Project-specific gap

The repository documents an end-to-end implementation but does not provide an independent, production-like test set, per-class precision and recall, a confusion matrix, score calibration, or a recorded field evaluation. The report treats those omissions as evaluation boundaries rather than filling them with invented results.

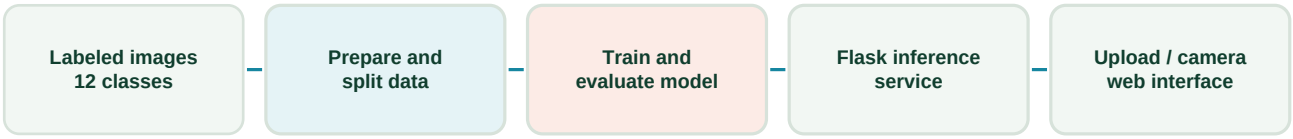
04 | System Overview

4.1 Existing approach

Without a visual decision aid, a user relies on prior knowledge, packaging labels, or separate local disposal information. Manual lookup can be slow, and material labels do not necessarily describe the local collection rules. A general-purpose visual classifier can offer a quick category suggestion, but it may be uncertain or outside its training domain.

4.2 Proposed system

EcoSort AI combines a labeled image dataset, a trained Keras model, a Flask inference service, and a responsive browser interface. The front end accepts a photo or an explicitly captured camera image. The backend validates the requested model and image input, transforms the image to the expected input size, executes the model, and returns the leading category candidates. The UI shows the top result and ranked alternatives.



4.3 Functional requirements

- Display available saved models and allow a valid model selection.
- Accept supported image file formats and reject missing or unsupported uploads.
- Capture a camera still, show it for review, and submit only after the user requests prediction.
- Return a top label and up to five ranked model scores.
- Show status and error feedback for camera and prediction actions.
- Adapt the layout to mobile-sized screens and expose install metadata for supported browsers.

4.4 Non-functional requirements

The interface should be understandable, responsive, and usable with current browsers. Prediction latency depends on the hosting CPU, memory, model loading, and network. Camera access requires a secure context such as localhost or HTTPS. For public use, service reliability, privacy communication, rate limiting, monitoring, and resource sizing require additional engineering.

05 | Dataset Description

The repository contains a folder-based image dataset, with class names used as labels. The saved model metadata records twelve categories and a total of 15,515 images. The counts below are read from models/metadata.json and sum to the recorded total.

5.1 Class counts

Class	Images	Share
Battery	945	6.1%
Biological	985	6.3%
Brown glass	607	3.9%
Cardboard	891	5.7%
Clothes	5,325	34.3%
Green glass	629	4.1%
Metal	769	5.0%
Paper	1,050	6.8%
Plastic	865	5.6%
Shoes	1,977	12.7%
Trash	697	4.5%
White glass	775	5.0%
Total	15,515	100.0%

5.2 Class distribution and imbalance

The clothes category accounts for approximately 34.3% of the images, while brown glass and green glass each contain fewer than 650 examples. This imbalance can influence optimization and class-specific performance. The training notebook computes class weights from class counts and passes them into model fitting to increase the contribution of less frequent classes. Weighting is a mitigation strategy; it does not create new examples or guarantee balanced performance.

5.3 Dataset provenance and quality

The repository snapshot does not document a complete dataset source, collection protocol, license, consent basis, deduplication report, or label-review process. These details should be recorded before redistribution or use beyond coursework. The folder structure suggests a single dominant label per image, but the report does not assume that every image has been manually audited. Lighting, camera viewpoint, image background, object condition, and near-duplicate images are relevant sources of variation.

The validation split is configured with seed 42 for reproducibility. A fixed seed makes the split repeatable under the same loader and data ordering; it does not by itself establish that the split is stratified or free of near duplicates.

06 | Tools and Technologies

Component	Use in the project
Python 3.11.9	Pinned runtime for compatibility with the TensorFlow deployment environment.
TensorFlow / Keras	Dataset loading, model definition, transfer learning, training, and inference.
MobileNetV2	ImageNet-pretrained convolutional backbone used for feature extraction.
NumPy	Numerical arrays and batch preparation for model prediction.
Pillow	Image decoding, RGB conversion, and resizing in the prediction service.
Flask	HTTP routes, form handling, JSON camera endpoint, and template rendering.
HTML / CSS / JavaScript	Responsive interface, upload controls, camera permission, capture, and results display.
Gunicorn	WSGI server command configured for hosted execution.
Render configuration	render.yaml and Python version files describe a hosted web-service setup.
ReportLab / Matplotlib artifacts	Training plot and project report figures; not part of runtime inference.

6.1 Runtime compatibility

The repository pins Python 3.11.9 through .python-version and runtime.txt, and render.yaml sets PYTHON_VERSION to 3.11.9. This was added to avoid a deployment selecting a newer Python release for which the configured TensorFlow dependency may not have a compatible distribution. Package constraints in requirements.txt include TensorFlow, NumPy, Matplotlib, Pillow, Flask, and Gunicorn.

6.2 Hardware and software context

Training and inference are resource-dependent. The repository does not specify a fixed GPU, RAM, or CPU requirement, nor does it record a benchmark latency for a named host. The model files are stored under models/. A deployment should be sized and load-tested for its expected concurrency; each worker process may need its own model memory.

07 | Data Preprocessing

7.1 Dataset loading and split

The training notebook uses a directory-based TensorFlow image loader. Class folders become label categories. The data is divided into training and validation portions with an 80/20 split and random seed 42. Images are batched in groups of 32, and dataset prefetching is used to overlap input work with model execution.

7.2 Image normalization and sizing

The training input is RGB with spatial dimensions of 224 by 224 pixels. At inference, the Flask service opens the uploaded or captured image with Pillow, converts it to RGB, resizes it to the dimensions specified in metadata, converts it to a TensorFlow image array, and adds a batch dimension before prediction. Matching these dimensions and channel conventions is necessary for compatibility with the trained model.

7.3 Training augmentation

The notebook applies random horizontal flip, rotation up to 0.08, zoom up to 0.12, and contrast adjustment during training. Augmentation exposes the model to modest image variations without changing the user-facing captured file. The validation pipeline is used to monitor generalization and does not serve as a real-world test set.

7.4 Class weighting

The saved metadata records one weight per class, with larger weights assigned to some less frequent classes and a smaller weight for clothes, the largest class. This changes the relative contribution of training examples in the loss. It should be reviewed alongside class-level evaluation because aggregate validation accuracy can hide weak performance on minority labels.

7.5 Data preparation summary



08 | Methodology

8.1 End-to-end methodology

The work proceeds from data inspection to model training, evaluation, application integration, and deployment packaging. Class counts are inspected from metadata; the notebook constructs repeatable train and validation datasets; MobileNetV2 is adapted to the twelve target labels; saved metrics and curves are reviewed; the chosen .keras model and metadata are loaded by Flask; and the browser interface supplies the image workflows.

8.2 Model development sequence

- Load labeled images from class directories and create the training and validation partitions.
- Apply resizing and training-only augmentation, and calculate class weights.
- Load MobileNetV2 with pretrained ImageNet weights and omit its original classification top.
- Freeze the backbone and train a task-specific classifier head.
- Unfreeze the final thirty backbone layers and fine-tune at a lower learning rate.
- Monitor validation behavior, stop or adjust learning rate as configured, and save the best checkpoint.
- Record class names, counts, image dimensions, class weights, and validation metrics in metadata.

8.3 Evaluation approach

The available quantitative evaluation is the saved validation accuracy and validation loss in models/metadata.json. The training-history plot provides epoch-level accuracy and loss curves. The repository does not report an independent held-out test score or a class-by-class confusion analysis, so conclusions are limited to the validation split and saved artifacts.

8.4 Application integration

The web application loads the ordered class labels from metadata, selects an available model file, preprocesses each image according to the same input dimensions, and maps model output indices back to class names. The camera API and upload route both use the same image-prediction function, reducing divergence between those input paths.

09 | Model Architecture and Training

9.1 Architecture



9.2 Classification head

The model uses MobileNetV2 without its original top layer as a feature extractor. The notebook adds global average pooling, dropout with rate 0.25, and a dense output layer with twelve units and softmax activation. The softmax values are sorted to select the leading candidate classes. The server returns up to five results.

9.3 Optimization and training stages

Training element	Configured approach
Initial stage	Train the new classification head while the backbone is frozen, for up to 12 epochs.
Fine-tuning stage	Unfreeze the final 30 MobileNetV2 layers and train for up to 5 additional epochs.
Optimizer	Adam; a lower learning rate of 1e-5 is used during fine-tuning.
Loss	Sparse categorical cross-entropy for integer class labels.
Early stopping	Monitor validation loss, patience 4, restore best weights.
Learning-rate scheduling	Reduce learning rate by factor 0.3 after two validation-loss plateaus, with a minimum of 1e-6.
Checkpoint	Save best_garbage_classifier.keras when validation accuracy improves.

9.4 Training artifacts

The model directory contains best_garbage_classifier.keras, garbage_classifier_final.keras, metadata.json, and training_history.png. Keeping the class ordering and image size in metadata helps the web application preprocess and interpret model output consistently.

10 | Web Application Implementation

10.1 Application structure

The Flask application is defined in app.py. The root route supports GET and POST: GET renders the interface, while POST validates an uploaded file and selected model, calls prediction, and returns the page with the uploaded image, top class, and score list. A separate POST /api/predict-frame route accepts a JSON model name and captured image data URL for camera prediction. A service-worker route serves the static worker script.

10.2 Upload path

- The user selects an available .keras model and a supported image file.

- The server requires a file, validates its filename extension, and checks that the selected model is in the available model list.

- Pillow decodes the image; the prediction helper converts it to RGB and resizes it to metadata dimensions.

- TensorFlow runs inference and the server sorts the class scores.

- The page shows the submitted image, the top label, its score, and up to five alternatives.

10.3 Camera path

- The browser requests video-only camera access and prefers the rear-facing camera where supported.

- The live preview is shown in the camera area.

- Capture photo copies the current frame to a canvas and scales it to at most 640 pixels wide.

- The captured still is displayed for review. Retake returns to the preview.

- Only Predict image sends the captured JPEG data URL and chosen model to /api/predict-frame.

- The server decodes the data URL, predicts, and returns the top class and score list as JSON.

This flow is user-triggered still-image classification. The camera preview itself is not continuously sent for inference. The source code stops camera tracks when the user stops the camera or switches away from camera mode.

11 | System Architecture

11.1 Component responsibilities

Layer	Responsibilities
Browser interface	Image upload, model selection, camera permissions, preview, capture and review controls, results and status messages.
Flask request layer	Route handling, form and JSON validation, model allow-list checks, request-size limits, and response generation.
Image preprocessing	Pillow decoding, RGB conversion, and resize to model metadata dimensions.
Inference service	Cached Keras model loading, probability calculation, top-five sorting, and label mapping.
Saved artifacts	Keras model files plus JSON metadata containing label order, input size, counts, weights, and metrics.
Hosting process	Gunicorn serves the Flask WSGI application; Render configuration supplies build and runtime commands.

11.2 Request flow



11.3 Data exchange

For uploaded photographs, the browser posts a multipart form to the root route. For camera capture, JavaScript serializes the reviewed still as a JPEG data URL and posts JSON containing the selected model name and image. The camera endpoint verifies that the selected model exists and that the image field is a string before decoding it. The prediction response contains a top_prediction object and a predictions list.

12 | User Interface Screenshots

The figures below are interface previews reconstructed from the repository's HTML and JavaScript controls. They document the implemented design and workflow but are not screenshots captured from a running Flask/TensorFlow server. The example object image is from the project's dataset.

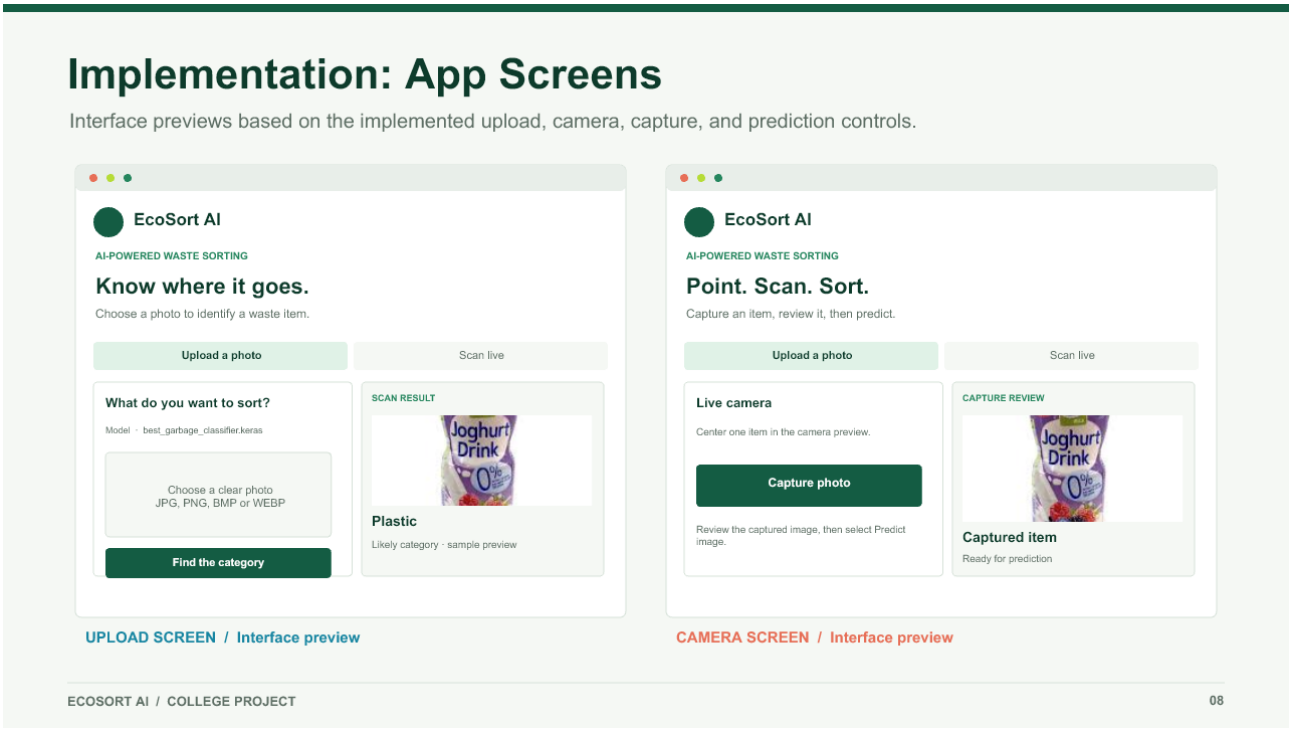


Figure 12.1 EcoSort AI upload and camera interface previews. Upload flow includes model selection and category action; camera flow shows capture, review, and prediction states.

12.1 Interface behavior

The page presents two modes, Upload a photo and Scan live. The upload mode includes model selection, a drop zone, and a result panel. Camera mode exposes Start camera, Stop camera, Capture photo, Retake, and Predict image actions. Status messages communicate camera and analysis state. The interface shows alternative score bars so users can see more than the single leading label.

12.2 Mobile and accessibility considerations

The layout uses responsive CSS and the app manifest defines standalone display metadata and an application icon. Native form controls, button labels, status roles, and accessible result regions are present in the template. A browser camera permission can be declined; upload remains an alternative. Installation and camera behavior depend on browser support and a secure origin.

13 | Results and Discussion

13.1 Saved validation metrics

Metric	Saved value	Interpretation
Validation accuracy	94.68%	Fraction of examples classified correctly on the recorded validation evaluation.
Validation loss	0.1720	Average loss reported for the selected saved evaluation.
Output categories	12	Number of configured class labels.
Dataset size	15,515	Count recorded in the class-count metadata.

13.2 Training history

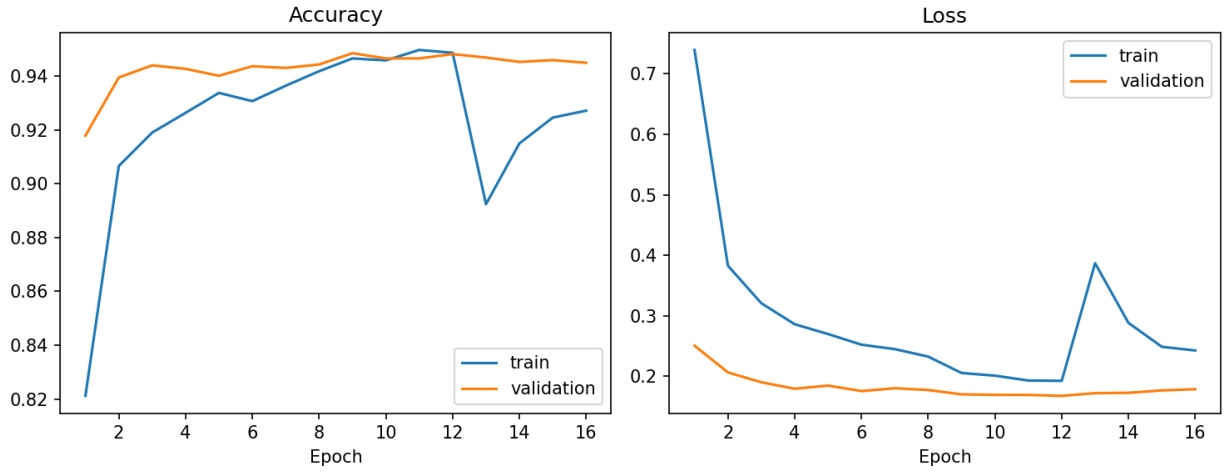


Figure 13.1 Saved training and validation accuracy/loss curves. The plot is a training diagnostic, not an independent test report.

The selected model metadata reports validation accuracy 0.9468256 and validation loss 0.1720355. The values in the table are rounded for presentation. The result is a useful project benchmark for the chosen split, but it cannot establish performance across unseen capture conditions or other populations of waste images.

13.3 Interpretation and evidence limits

- Overall accuracy can hide errors concentrated in smaller classes such as brown glass or green glass.

- The repository does not include a confusion matrix or per-class precision, recall, or F1 values in the saved metadata.

- The 80/20 validation split is not documented as a separate, independently curated test set.

- The score shown by the app is a model output and has not been documented as calibrated confidence.

- No latency, throughput, or deployment load-test result is recorded in the project files.

13.4 Result conclusion

The project demonstrates a functioning design path from labeled images and transfer learning to a browser-based prediction workflow. The available evidence supports reporting the saved validation metrics and the implemented system features. Broader claims about real-world accuracy, safety, or local recycling outcomes require additional controlled evaluation.

14 | Verification and Test Plan

The project repository includes application and model code but does not contain a formal test report with recorded execution results. The table below is a recommended verification matrix for the college demonstration and future release; it should not be read as a claim that every test has been executed.

Test area	Test case	Expected behavior	Evidence status
Upload	Submit a supported image with a valid model.	Show top label, submitted image, and ranked scores.	Recommended; execution not recorded
Upload validation	Submit no image or an unsupported extension.	Show a helpful error and do not run inference.	Recommended; execution not recorded
Camera permission	Start camera on HTTPS or localhost and allow access.	Display the live preview and camera status.	Recommended; execution not recorded
Capture and review	Capture, inspect, then retake.	Show still image; retake restores preview.	Recommended; execution not recorded
Prediction action	Predict only after capture and user confirmation.	Send one still and show returned ranked labels.	Recommended; execution not recorded
Model selection	Select a missing or invalid model through API request.	Reject the request with a client error.	Code path present; execution not recorded
Mobile layout	Open at narrow phone viewport.	Controls remain readable and usable without overlap.	Recommended; execution not recorded
Deployment	Install requirements and start Gunicorn with pinned runtime.	Service starts and responds over HTTPS.	Config present; deployment status not verified

14.1 Acceptance criteria for a demonstration

- The selected model appears in the UI and one known sample image yields a non-empty result.
- The camera screen clearly separates preview from captured-image review and prediction.
- Unsupported inputs produce a message rather than an unhandled UI failure.
- Results are described as model suggestions and the validation/test distinction is stated.
- The report, presentation, and demonstration use consistent project facts and metrics.

15 | Deployment and Operation

15.1 Runtime configuration

Setting	Repository configuration
Python runtime	3.11.9 in .python-version, runtime.txt, and render.yaml.
Build command	pip install -r requirements.txt
Start command	gunicorn --bind 0.0.0.0:$PORT app:app
Application module	app.py; Flask application object is named app.
Model storage	models/*.keras and models/metadata.json.
Upload body limit	Flask MAX_CONTENT_LENGTH is 8 MB.
Camera encoded input	Data URL is rejected when encoded input length exceeds the configured threshold.

15.2 Local execution

A local setup should use Python 3.11, create an isolated virtual environment, install requirements.txt, and run python app.py. The application binds to localhost and normally serves on port 5000 unless PORT is set. Camera access is expected to work on localhost. A hosted camera workflow requires HTTPS.

15.3 Hosting notes

The repository contains Render-compatible configuration. A successful build depends on the host honoring the pinned Python version and installing compatible TensorFlow packages. The configuration files describe intended deployment; their presence alone does not prove that a public service is currently live. Hosting memory, worker count, startup time, and inference latency should be measured on the selected service plan.

16 | Privacy, Security, and Operational Risks

16.1 Current safeguards in code

- The upload route checks that a file is supplied and restricts filename extensions.
- The selected model name is checked against model files available in the configured models directory.
- The Flask request body is limited to 8 MB.
- The camera endpoint validates the model selection and image data URL shape and size before decoding.
- The project code does not define user accounts, upload libraries, or a database for storing user photos.

16.2 Remaining production work

Extension checks are not equivalent to full content inspection. A public service should validate actual MIME and decoded image properties, cap dimensions and processing time, apply rate limits, monitor errors, and define a privacy notice. Request and model-loading behavior should be checked under concurrent load. A deployment should avoid exposing debug mode and should use managed secrets if external services are added.

16.3 User-facing risk

A visual label does not establish recyclability, cleanliness, or local disposal eligibility. Battery and hazardous-waste categories require especially careful local instructions. The interface should not imply that the model is infallible or that a confidence score is a verified probability. A low-confidence or unknown response is a recommended improvement.

17 | Advantages and Limitations

17.1 Project advantages

- The app integrates model inference with both photo upload and camera capture workflows.

- Transfer learning reuses pretrained visual features rather than training an entire feature extractor from scratch.

- The camera flow gives users a visible preview and review step before an image is submitted.

- Class weights address the strongly uneven folder counts during training.

- The PWA metadata and responsive layout support a mobile-oriented browser experience.

- The model, metadata, training plot, runtime configuration, and application code are organized in the project repository.

17.2 Current limitations

- The classifier predicts one of twelve labels and cannot localize multiple objects in a mixed scene.

- Dataset provenance, licensing, collection conditions, and independent label review are not documented in the repository.

- Clothes makes up about one third of the images; class weighting cannot guarantee equal class performance.

- Only aggregate validation metrics are saved in metadata; per-class metrics and a confusion matrix are absent.

- No independent held-out test result, robustness study, or calibration analysis is reported.

- The PWA can cache the web shell, but TensorFlow inference remains server-side and needs a network connection.

- The app offers material categories, not validated local recycling instructions.

18 | Future Scope

18.1 Model and dataset improvements

- Document source, license, labeling criteria, and collection conditions for each dataset subset.

- Create a separate held-out test set with class-wise precision, recall, F1, confusion matrix, and error review.

- Collect additional examples for minority categories and varied phones, lighting, backgrounds, viewpoints, and item conditions.

- Assess duplicate leakage and consider grouped or stratified splitting where dataset structure allows.

- Evaluate score calibration and add an explicit low-confidence or unknown-object outcome.

- Explore object detection or segmentation if the intended use includes multiple objects in a frame.

18.2 Application improvements

- Add request rate limiting, image dimension limits, health checks, structured logging, and service monitoring.

- Test deployment memory, cold-start time, and inference latency under expected traffic.

- Improve accessibility and local-language support with user testing.

- Connect disposal guidance only to maintained local rules, with source and jurisdiction displayed.

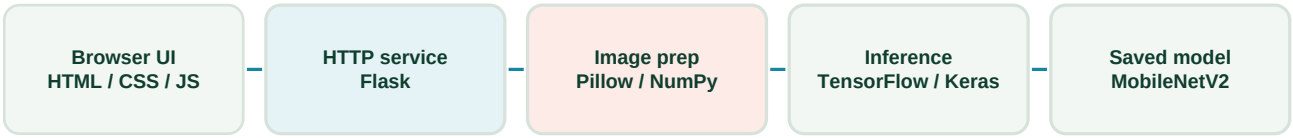
- Evaluate model optimization or on-device inference only after benchmarking supported devices and reviewing privacy implications.

18.3 Evaluation roadmap

A follow-on evaluation should define the target population and capture conditions before collecting test images. The test set should remain separate from training and model selection. Results should report overall and class-wise metrics, uncertainty or confidence behavior, error examples, and the conditions under which the model should abstain. User studies should assess whether the interface helps people sort correctly rather than merely whether they can operate the app.

19 | Software Stack at a Glance

EcoSort AI is a small full-stack machine-learning system. Its components are separated by responsibility: Python hosts the model and HTTP service; TensorFlow/Keras defines and executes the neural network; NumPy and Pillow prepare image data; Flask connects HTTP requests to inference; and browser technologies provide the user workflow. Deployment files describe how a compatible runtime can install and start the service.



19.1 Component boundaries

Layer	Project technology	Responsibility
User interface	HTML, CSS, JavaScript	Image selection, camera preview, still capture, request, and result display.
Web service	Flask	Routes, request validation, template rendering, and JSON response.
Image handling	Pillow, NumPy	Decode, convert, resize, and form the model input batch.
Machine learning	TensorFlow / Keras	Load the saved network and calculate ranked class scores.
Runtime / deployment	Python 3.11.9, Gunicorn, Render configuration	Dependency installation and hosted WSGI process configuration.

The boundaries matter when debugging. A camera permission failure belongs to the browser workflow; a malformed upload is an HTTP/input issue; an import or incompatible wheel is a runtime issue; an incorrect class name may be a metadata-order issue; and a confusing prediction may be a model, image-quality, or domain-shift issue. Treating every failure as an AI problem makes diagnosis harder.

20 | Python Runtime and Dependency Management

20.1 Python's role

Python is the common runtime for the training notebook and Flask inference service. In the notebook it orchestrates dataset loading, augmentation, model definition, optimization, validation, and artifact export. In app.py it accepts an image, uses the saved model, and converts model outputs into a response the web interface can display. Python also provides the ecosystem in which TensorFlow, Pillow, NumPy, Flask, and Gunicorn interoperate.

20.2 Runtime pin and the deployment build failure

The repository specifies Python 3.11.9 in .python-version, runtime.txt, and render.yaml. This is important because TensorFlow publishes binary packages for supported Python/platform combinations rather than every possible version. A service that selects a newer Python release may fail while resolving the TensorFlow requirement before the application can start. The project configuration is intended to constrain that runtime; the exact hosted build must still be verified after deployment.

20.3 Requirements and reproducibility

requirements.txt records minimum-version constraints for TensorFlow, NumPy, Matplotlib, Pillow, Flask, and Gunicorn. Minimum constraints permit a range of later releases, so they are not a fully locked dependency solution. For repeatable builds, a future release should record a tested lock file or exact compatible versions, target platform, and installation command. Model and Python compatibility should be validated together.

A Python virtual environment isolates packages from other projects. It reduces accidental conflicts but does not fix an unavailable wheel, a wrong interpreter version, insufficient memory, or a model serialization incompatibility. The useful diagnostic sequence is to inspect the selected Python version, platform, package index output, and the first dependency-resolution error.

21 | TensorFlow and Keras

21.1 Framework responsibilities

TensorFlow is the numerical and deep-learning framework used by the project. Keras supplies the higher-level model-building API used in the notebook: a pretrained application backbone, augmentation layers, pooling, dropout, a dense classification layer, model compilation, training callbacks, evaluation, and saved-model loading. The inference web service reuses the exported model rather than rebuilding its architecture in the request handler.

21.2 Dataset input and batching

The notebook uses image_dataset_from_directory with integer labels, a 224 by 224 target size, batch size 32, validation_split 0.2, seed 42, and explicit training/validation subsets. Directory names supply class labels. Batching groups examples for efficient accelerator or CPU processing. AUTOTUNE prefetch overlaps data preparation with model execution where possible.

21.3 Saved model and metadata contract

The model artifact and metadata file form a contract. The model expects three-channel image input at the trained spatial resolution and produces twelve scores. metadata.json supplies the class ordering, image size, class counts, weights, and recorded validation values. If model output indices and metadata class_names drift out of alignment, the interface can display a plausible but incorrect label. Both artifacts should therefore be released and tested as one versioned unit.

21.4 Inference behavior

The backend loads a Keras model through a cached loader and calls predict on a one-image batch. It sorts the output and returns up to five candidates. This is a single-label classifier's ranking, not a detection list of every object in a scene. A multi-item photograph may receive one dominant class even though several kinds of material are visible.

22 | MobileNetV2 Architecture

22.1 Why this backbone

MobileNetV2 is a convolutional network designed for efficient visual feature extraction. Its inverted residual blocks expand channels, apply depthwise spatial convolution, and project back through a narrow linear bottleneck. Skip connections are used where tensor dimensions allow. The design reduces computation relative to a conventional dense convolution at comparable feature-map sizes [1].

22.2 Pretrained representation

The notebook requests MobileNetV2 with include_top=False and ImageNet weights. Excluding the original classifier leaves a feature map suitable for a new task-specific head. ImageNet pretraining provides generic visual features learned from a large image collection; the waste dataset then adapts the representation to its twelve directory labels. This is a transfer-learning choice, not evidence that ImageNet contains the project's exact waste categories.

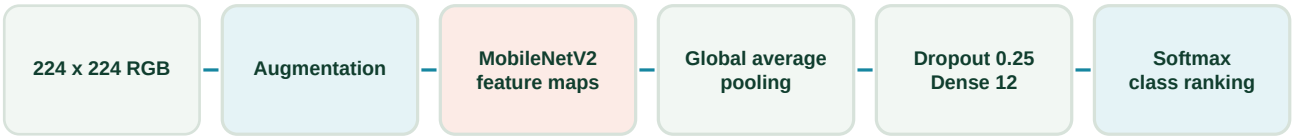
22.3 Input scaling

The application expects 224 by 224 RGB images. MobileNetV2's preprocess_input maps pixel values to the range expected by this architecture. In the notebook, that preprocessing layer is included in the saved model graph. The serving code therefore converts and resizes the image but should not apply an additional incompatible normalization a second time. Consistency between training and serving prevents a subtle but damaging train/serve mismatch.

22.4 Mobile architecture versus mobile execution

Although the backbone is designed for efficiency, the current project runs prediction in the Flask server process. The browser sends the captured or uploaded image over the network. It does not bundle TensorFlow Lite or execute the model locally on the phone. Offline inference would require a separate conversion, on-device integration, performance testing, and model licensing/compatibility review.

23 | Classifier Head and Transfer-Learning Stages



23.1 Classification head

The input passes through the augmentation sequence and MobileNetV2 preprocessing before the pretrained backbone. GlobalAveragePooling2D compresses each feature channel to a scalar, reducing the number of parameters compared with a large flattening layer. Dropout at 0.25 regularizes the new head during training. A Dense layer with twelve softmax outputs maps the learned representation to the configured classes.

23.2 Frozen-backbone training

The first stage freezes the pretrained backbone and trains the newly initialized classification head. This lets the head learn the target label mapping while retaining the pretrained visual features. The notebook compiles with Adam and sparse_categorical_crossentropy, which is appropriate for integer-encoded class labels. Training is configured for up to twelve epochs, subject to early stopping and checkpoint behavior.

23.3 Fine-tuning

The second stage unfreezes the final thirty backbone layers and reduces the learning rate to 1e-5. A small fine-tuning rate helps adapt higher-level visual features without making large destructive updates to the pretrained representation. The notebook allows training up to seventeen total epochs and combines the histories from the two stages for plotting.

The training-history figure shows a visible disturbance around the fine-tuning transition. This should be interpreted by inspecting the stage boundary, optimizer state, train/validation curves, and checkpointed best weights. The saved validation metrics identify the selected recorded artifact; they do not remove the need to audit training behavior.

24 | Augmentation and Class Imbalance

24.1 Training augmentation

The model applies random horizontal flipping, rotation up to 0.08, zoom up to 0.12, and contrast variation up to 0.1. These transformations are intended to expose the classifier to modest changes in orientation, framing, and illumination. Augmentation is active in training and is designed to be inactive for ordinary inference. It can improve robustness to some variations, but cannot substitute for collecting representative data or correcting mislabeled examples.

24.2 Class weights

The notebook derives a weight for each class from total_images divided by the product of the number of classes and that class's image count. Consequently, classes with fewer examples receive larger per-example weight during training. This changes the contribution to the loss; it does not alter validation examples or add images.

24.3 Interpreting the distribution

Clothes has 5,325 of 15,515 images (about 34.3%), while brown glass has 607 and green glass 629. Shoes is the next largest group at 1,977. Class weighting is a reasonable response to this imbalance, but final behavior must be measured class by class. In particular, aggregate accuracy can conceal poor recall for smaller groups.

24.4 Data hygiene recommendations

Before retraining, document dataset source and permission, identify duplicate or near-duplicate files, review label conventions, and inspect representative errors. If related images originate from the same object, scene, or capture session, group-aware splitting can reduce leakage. Any proposed re-split should be accompanied by newly calculated validation metrics rather than compared as though it were the same experiment.

25 | Training Controls and Reproducibility

25.1 Optimization

Adam updates model weights using gradient information with adaptive per-parameter step sizes. The notebook uses sparse categorical cross-entropy as the loss and accuracy as a monitored metric. Class weights are supplied to the fitting process. A training seed helps reproduce the dataset split and related random operations, but exact numerical reproducibility can still depend on TensorFlow, hardware, kernel implementations, and software versions.

25.2 Callbacks

Callback	Configured behavior	Purpose
ModelCheckpoint	Save best_garbage_classifier.keras; monitor val_accuracy; save_best_only; mode max	Retain the checkpoint with the strongest monitored validation accuracy.
EarlyStopping	Monitor val_loss; patience 4; restore_best_weights true	Stop when validation loss does not improve and restore selected weights.
ReduceLROnPlateau	Monitor val_loss; factor 0.3; patience 2; minimum learning rate 1e-6	Reduce the learning rate when validation loss plateaus.

25.3 Artifact reporting

The notebook evaluates the final model on the validation dataset, saves the Keras artifact and metadata, and plots training history. The metadata reports validation accuracy 0.9468256 and validation loss 0.1720355. These are useful project-reported validation values. The repository snapshot does not include an independently held-out test set, per-class precision/recall, confidence calibration, or repeated-run variance, so those measurements are not claimed here.

26 | NumPy, Pillow, and Image Preparation

26.1 Pillow decoding

Pillow opens uploaded image content and provides RGB conversion and resizing. Converting to RGB avoids mode differences such as grayscale or palette images causing an unexpected channel shape. Resizing to metadata's configured 224 by 224 dimensions aligns user images with the network input. Content decoding can still fail for corrupt, truncated, oversized, or unsupported files; a public service should handle those failures cleanly.

26.2 NumPy batch shape

The decoded image is converted to an array and expanded with a batch axis. A single RGB image therefore has a batch shape conceptually equivalent to (1, 224, 224, 3). TensorFlow models commonly accept a batch even when only one image is supplied. The serving pipeline should verify finite values, expected dimensions, and channel count before inference.

26.3 Training/serving parity

Preprocessing parity is crucial: the same color interpretation, resize convention, numeric range, and architecture-specific input scaling should be applied at training and inference. In this project MobileNetV2 preprocessing is part of the saved model. Reapplying a generic 0-to-1 scale outside the model could change the expected input range and lower accuracy. The backend's responsibility is therefore deliberately limited to reliable decode, RGB conversion, resize, array creation, and batch expansion.

27 | Flask Backend and Prediction API

27.1 Web routes

Flask is the lightweight Python web framework used to connect browser requests to the model. The root route renders the interface and handles the form workflow. The camera capture path sends a data URL to /api/predict-frame; the endpoint validates the model selection and image payload and returns JSON containing the top prediction and ranked candidates. Flask's request size limit is configured to 8 MB.

27.2 Model loading and reuse

The model discovery helper checks available .keras files. The loader is cached with a maximum of four model instances, avoiding a fresh disk load for every prediction. This improves repeated inference but makes memory use dependent on the number and size of cached models. A production service should confirm cache behavior, startup time, and worker memory under its actual hosting configuration.

27.3 Input validation

The camera endpoint checks the data URL's image prefix, delimiter, and encoded length before base64 decoding. Image extensions are checked in the upload workflow and invalid model choices are rejected. These are useful application checks, but robust public deployment should also validate decoded content, pixel dimensions, MIME/content consistency, execution time, and request rates.

27.4 Response design

The model scores are sorted and the top five are returned. The top label is a useful interface summary, while alternatives communicate some ambiguity. Scores should be named as model scores or confidence estimates rather than guaranteed correctness probabilities unless calibration is evaluated. A future response may include a low-confidence/unknown state and a request identifier for diagnostics without storing personal images.

28 | Browser Interface: HTML, CSS, and JavaScript

28.1 HTML structure

The template defines the page structure for image upload, live camera preview, capture, retake, prediction, and result display. Semantic controls and labels help users understand which action is available at each step. The server-rendered template also presents standard form errors and prediction results.

28.2 CSS and responsive layout

The stylesheet provides the visual identity, responsive layout, component states, and mobile-oriented presentation. Responsive design must preserve readable labels, tappable controls, and a stable camera preview at narrow widths. A screenshot documents a particular rendering, but browser testing at multiple viewport sizes remains necessary to establish that the layout behaves consistently.

28.3 JavaScript workflow

JavaScript requests camera access, attaches the media stream to a visible video element, draws the chosen frame to a canvas, converts that still to an image data URL, and submits it only after the user chooses Predict. Capture and retake make the review state explicit. This is image capture followed by prediction, not continuous video inference.

28.4 Error and permission states

Camera access depends on user permission, supported browser APIs, and a secure context. If permission is denied, no device is available, or the page is not served from localhost/HTTPS, the user should receive a clear alternative such as uploading a photo. The camera UI should also stop media tracks when the workflow ends so that the camera indicator turns off.

29 | Progressive Web App and Camera APIs

29.1 PWA manifest

static/manifest.webmanifest describes the app name, start URL, standalone display preference, theme colors, and icons. Compatible browsers can use this metadata to offer an installable experience. Installation behavior varies by browser and platform and should be checked on actual target devices.

29.2 Service worker

The service worker caches selected application-shell GET resources. This can make parts of the interface available more quickly and may help the shell reopen when connectivity is limited. It does not include the saved TensorFlow model or perform local inference. A prediction still requires the browser to reach the Flask service.

29.3 getUserMedia and still capture

The browser camera API supplies a live MediaStream for preview. The JavaScript capture action draws one frame to a canvas and sends that image to the backend when the user requests prediction. Browser permission is controlled by the user; camera access is normally available only on HTTPS or localhost. Device orientation and camera selection can vary and merit cross-device checks.

29.4 Privacy and lifecycle

The current repository does not define accounts or a database for uploaded photos. A release should still explain how requests are processed and whether hosting logs include request metadata. The interface should stop the stream on retake/close/navigation as appropriate, avoid accidental repeated submissions, and expose an upload fallback.

30 | Interface Screenshot: Upload Workflow

The following project-interface preview is derived from the implemented HTML/CSS interface and presentation capture. It illustrates the upload-oriented screen and is not a screenshot of a verified live hosted service.

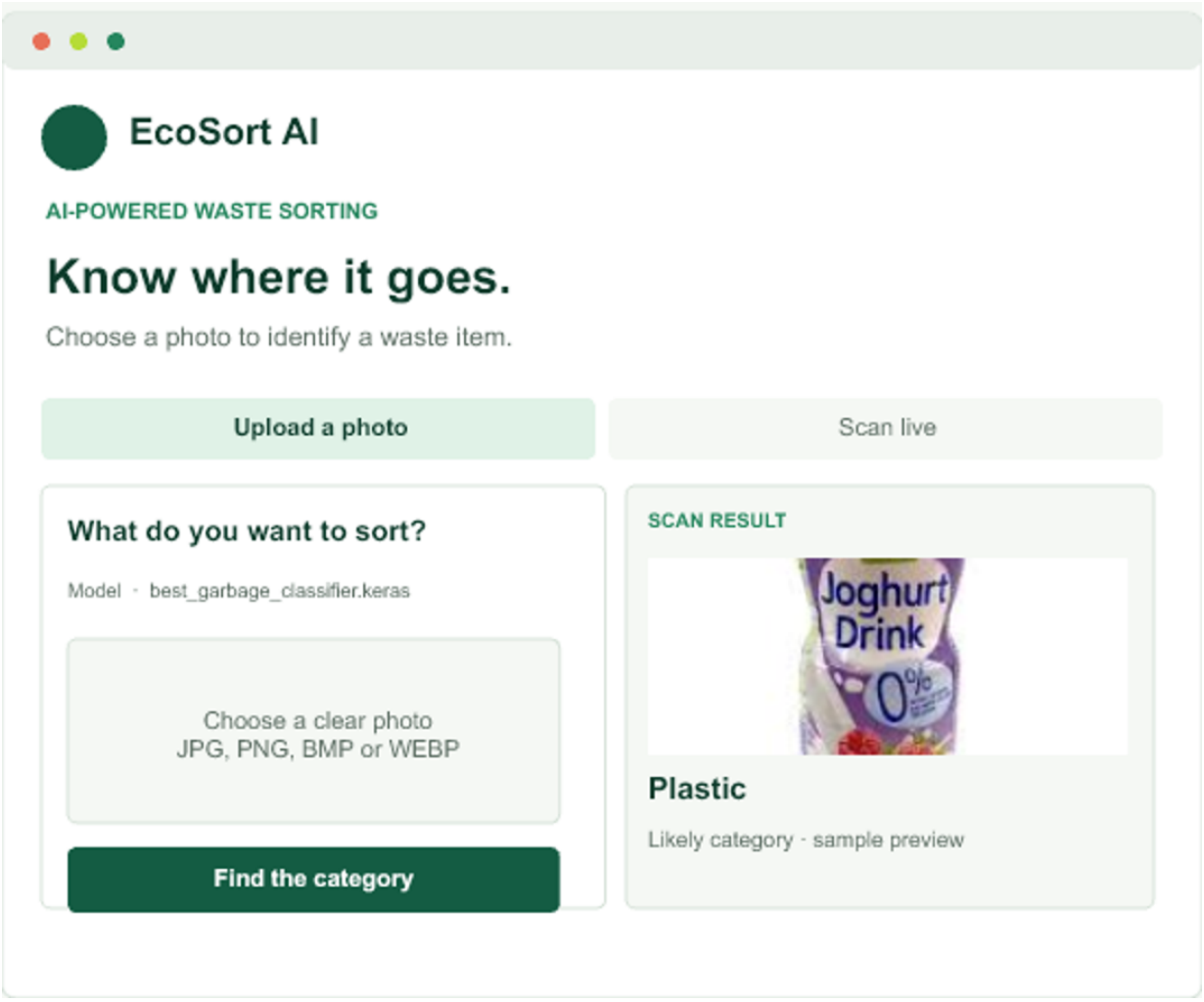


Figure 30.1. EcoSort AI upload screen preview. The user selects an image and requests server-side classification.

30.1 Expected user sequence

- Choose a supported image file from the device.
- Confirm the selected image is the intended item and is clearly visible.
- Submit the image to the Flask service for prediction.
- Review the top category and ranked alternatives as model suggestions.
- Use local disposal guidance for the final sorting decision.

31 | Interface Screenshot: Camera Capture

This camera-screen preview shows the live preview and capture-oriented controls represented in the project interface. It documents the intended browser workflow, not evidence that a camera was active in the screenshot or that a particular host deployment was tested.

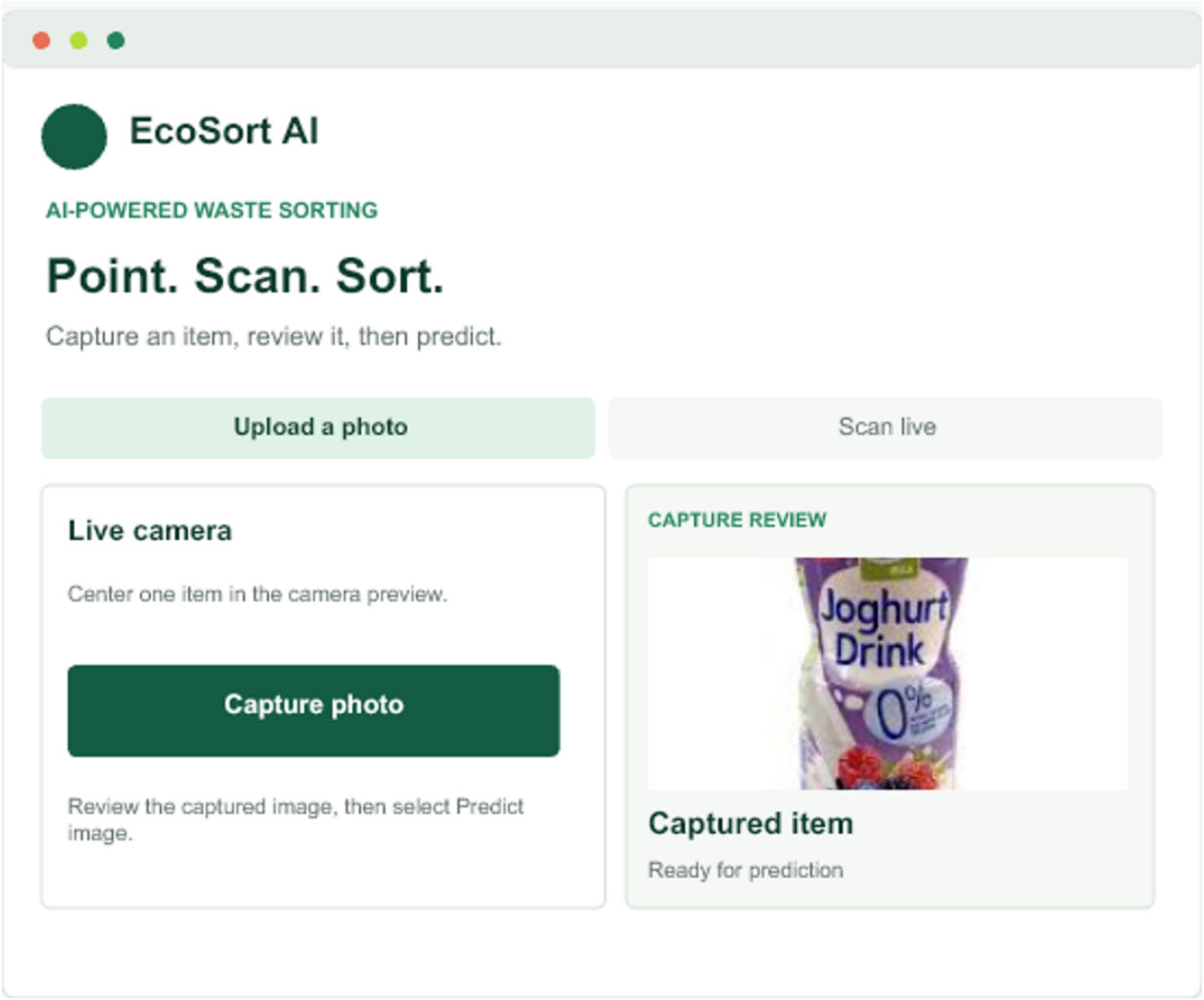


Figure 31.1. Camera workflow interface preview. Capture first, review the still image, then explicitly request prediction.

31.1 Interaction states

State	What the user sees	Next action
Permission pending	Camera start/status feedback	Allow access or choose upload.
Live preview	Visible camera image and capture action	Frame a single item and capture.
Still review	Captured image replacing/overlaid on preview	Predict or retake.
Prediction result	Top category and ranked alternatives	Review suggestion; check local rules.

32 | Dataset Sample Gallery

The figure below contains one representative image selected from each class directory in the repository. These are dataset examples, not app prediction screenshots. A small gallery helps readers understand the visual range and label vocabulary, but it is not a substitute for a dataset audit or a held-out test set.

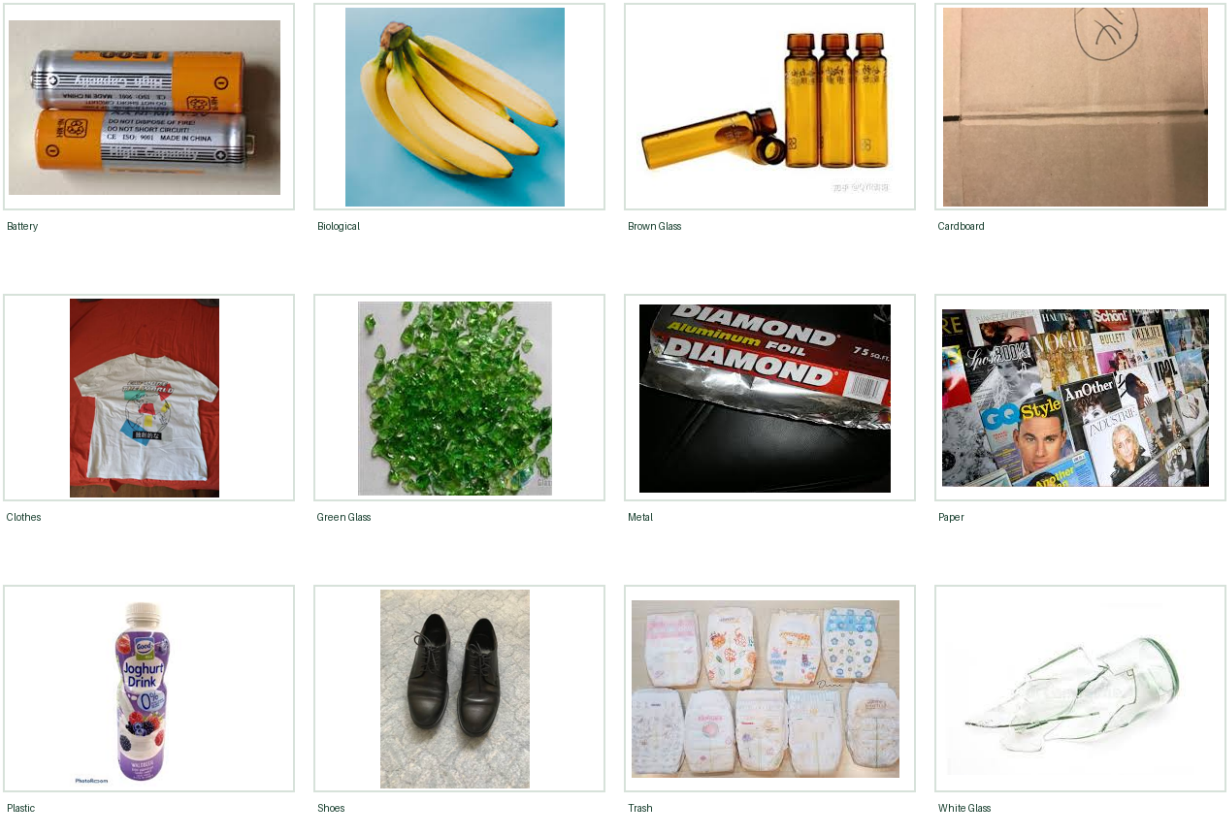


Figure 32.1. One repository sample per configured class: battery, biological, brown glass, cardboard, clothes, green glass, metal, paper, plastic, shoes, trash, and white glass.

32.1 Visual interpretation

Some categories represent material families (paper, metal, glass, plastic), while others describe object or use categories (clothes, shoes, battery) or a broad residual class (trash). These labels are not perfectly mutually exclusive descriptions of material composition. The learned target follows the dataset's folder labels; category definitions should be documented before interpreting model errors or extending the class list.

33 | Class Distribution and Results Interpretation

33.1 Recorded counts

The dataset contains 15,515 labeled images across twelve classes. Clothes represents about one third of the examples; shoes represents about 12.7%; each remaining category is below 7%. This structure explains why the training notebook computes class weights and why aggregate validation accuracy is an incomplete view of model quality.

Group	Count	Approximate share
Clothes	5,325	34.3%
Shoes	1,977	12.7%
Paper	1,050	6.8%
Biological	985	6.3%
Battery	945	6.1%
Other seven classes	5,233	33.7%
Total	15,515	100.0%

33.2 Validation result

The saved metadata records validation accuracy of 0.9468 and validation loss of 0.1720. Accuracy is the fraction of validation examples whose highest-scoring class matches the label. Loss captures the model's categorical error under the training loss definition. Neither statistic independently establishes performance on images captured by new users, and neither reveals which classes are confused.

33.3 Claims boundary

No result is reported for an independent test dataset because none is documented in the repository snapshot. The report also does not invent per-class precision, recall, F1, a confusion matrix, camera latency, or user-study outcomes. Such values should be added only after running and recording the corresponding evaluation protocol.

34 | User Guide: Photo Upload

34.1 Prepare a useful image

Choose a well-lit image with one item occupying a substantial part of the frame. Keep the item in focus, avoid covering its important shape, and use a background that does not visually merge with it. The app predicts from the pixels provided; it cannot read a recycling symbol or infer a local collection rule unless a separate feature is developed.

34.2 Submit and read the result

- Open the EcoSort AI page and choose the upload workflow.
- Select an allowed image file and verify that the preview matches the intended object.
- Submit once and wait for the server response.
- Read the highest-ranked category and any alternatives shown by the interface.
- Treat the scores as model output, not a guarantee; check local waste guidance.

34.3 If the result is poor

Retake the photo with better focus and lighting, simplify the scene, and ensure the object is large enough to recognize. If the object contains several materials or is outside the twelve classes, the classifier may still force a known category. A wrong result should be recorded with the image conditions for later evaluation, subject to the project's privacy policy.

35 | User Guide: Camera and Troubleshooting

35.1 Camera steps

- Open the site on HTTPS or localhost and choose camera scan.
- Allow camera permission when the browser prompts.
- Frame one item and confirm the live preview is visible.
- Capture a still image and inspect it before continuing.
- Choose Predict to submit that still; choose Retake to capture again.
- When finished, close the camera workflow so the media stream can stop.

35.2 Common issues

Symptom	Likely cause	Suggested action
Browser does not request camera	Insecure origin, permission already denied, or unsupported browser	Use HTTPS/localhost; review site permissions; try upload.
Black or blank preview	Camera busy, blocked, or no device	Close other camera apps; check permissions/device; refresh.
Prediction request rejected	Large/invalid payload or model unavailable	Try a smaller supported photo; confirm server model files.
Unexpected category	Ambiguous image, domain shift, or model limitation	Improve framing; compare alternatives; do not treat as definitive.

35.3 Responsible use

The tool should not be used to determine safe handling of hazardous materials or to override municipal instructions. Batteries and special waste need verified collection guidance. If uncertain, do not place an item in a recycling stream solely because of a model label.

36 | Deployment Environment and Compatibility

36.1 Configured service

Item	Project setting
Runtime	Python 3.11.9
Dependency installation	pip install -r requirements.txt
WSGI process	gunicorn --bind 0.0.0.0:$PORT app:app
Application object	app.py:app
Model files	models/*.keras and models/metadata.json
Request size	Flask request body maximum 8 MB
Camera origin	HTTPS required for hosted browser camera access

36.2 TensorFlow packaging concern

The earlier deployment log showed package resolution stopping at a TensorFlow lower-bound constraint because no compatible distribution was available for the selected environment. The repository now contains explicit Python 3.11.9 pins; a deploy must be built from the latest commit containing those files. A successful local installation does not prove a hosted build will succeed, because operating system, architecture, Python patch version, package index, and memory limits also matter.

36.3 Deployment verification

After publishing, record the commit hash, runtime reported by the host, dependency installation log, service health response, model load outcome, and a controlled test prediction. Camera access must be checked on the public HTTPS domain and on target phone browsers. The mere presence of render.yaml is configuration evidence, not proof that the service is live.

37 | Integration, Verification, and Traceability

37.1 End-to-end data path



37.2 Traceability matrix

Requirement	Implementation evidence	Verification status
Twelve-class prediction	Notebook, models/*.keras, models/metadata.json	Saved artifact/metadata present; independent test not recorded.
Photo upload	templates/index.html and Flask root route	Code path present; formal test run not recorded.
Camera still capture	JavaScript camera/canvas flow and /api/predict-frame	Implemented capture-then-predict flow; device matrix not recorded.
Class mapping	metadata.json class_names	Must match model output ordering; recommend automated check.
Mobile/PWA shell	CSS, manifest, service worker	Files present; cross-browser installation not recorded.
Hosted execution	render.yaml, runtime.txt, .python-version	Configuration present; current public status unverified.

37.3 Minimum release checks

A release should run a known-image smoke test, invalid-file test, missing-model test, camera permission/capture test, narrow-screen test, and hosted cold-start test. Test evidence should record software versions, image fixture identifiers, expected behavior, actual outcome, and date. Until that record exists, this report identifies these as recommended checks, not completed passes.

38 | Conclusion

EcoSort AI demonstrates an end-to-end image-classification prototype for waste categories. The project organizes 15,515 images into twelve class folders, applies transfer learning with MobileNetV2, and exposes inference through a Flask application. The web interface supports photo upload and a camera workflow that shows a live preview, captures a still, permits review, and predicts only after explicit user action.

The saved model metadata reports 94.68% validation accuracy and 0.172 validation loss. Those values are tied to the project validation split and should not be generalized to arbitrary real-world images. The repository does not provide an independent test benchmark or class-level performance report. This distinction is central to interpreting the result responsibly.

The principal outcome is an integrated academic prototype connecting dataset preparation, model training, inference, and an accessible browser experience. Before broader use, the work should improve dataset provenance, independent evaluation, minority-class analysis, low-confidence handling, deployment monitoring, and locally verified disposal information. Within those boundaries, the project provides a practical foundation for continued study of computer vision in waste sorting.

Final project outcome

A trained twelve-class image classifier integrated with a responsive web app for uploaded photos and user-captured camera stills, accompanied by documented validation results and a defined evaluation roadmap.

References

Project-specific statements in this report are based on the repository files listed below. External sources support the general technical background and framework descriptions.

[1] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 4510-4520. https://openaccess.thecvf.com/content_cvpr_2018/html/Sandler_MobileNetV2_Inverted_Residuals_CVPR_2018_paper.html

[2] TensorFlow, "tf.keras.applications.MobileNetV2 API," TensorFlow API documentation. https://www.tensorflow.org/api_docs/python/tf/keras/applications/MobileNetV2

[3] Pallets Projects, "Flask Quickstart," Flask documentation. https://flask.palletsprojects.com/en/stable/quickstart/

[4] EcoSort AI project repository, README.md; garbage_classification_model.ipynb; app.py; templates/index.html; static/styles.css; static/manifest.webmanifest; static/service-worker.js; models/metadata.json; models/training_history.png; requirements.txt; render.yaml; runtime.txt; .python-version. Local project snapshot, accessed 18 October 2026.

[5] Fleet Project report.pdf, supplied by the user. Used as a structural reference for academic front matter, chapter organization, results, screenshots, limitations, and appendices; its project content has not been reused.

39 | Appendices

Appendix A Team contribution plan

Team member	Proposed responsibility area	Example deliverables
Arshpreet Singh	Dataset and model development	Dataset review; MobileNetV2 training; class weighting; validation analysis.
Sarbjit Singh	Backend and integration	Flask routes; model loading; upload and camera API integration; deployment configuration.
Pooja	Interface, testing, and documentation	Responsive UI; capture-review workflow; interface checks; report and presentation.
All team members	Shared project work	Planning; integration review; demonstration preparation; final submission review.

The project files do not contain a contribution log. These responsibilities are proposed from the codebase work areas and should be confirmed by the team before submission.

Appendix B Repository map

Path	Purpose
app.py	Flask routes, image decoding, model loading, and inference.
garbage_classification_model.ipynb	Notebook for data loading, model training, and evaluation.
garbage_classification/	Folder-based labeled image dataset.
models/metadata.json	Class names, image dimensions, class counts, class weights, and saved validation metrics.
models/*.keras	Saved trained Keras model artifacts.
models/training_history.png	Training and validation accuracy/loss plot.
templates/index.html	Upload, camera, capture/review controls, and prediction-result UI.
static/styles.css	Responsive styles and branded interface presentation.
static/manifest.webmanifest	PWA name, start URL, display mode, theme, and icon metadata.
static/service-worker.js	Application shell caching and GET request cache behavior.
requirements.txt	Python dependency constraints.
render.yaml, runtime.txt, .python-version	Hosted service settings and Python runtime pin.

Appendix C Reproducible setup and remaining submission details

1. Install Python 3.11 and create a project virtual environment.
2. Install dependencies from requirements.txt.
3. Confirm models/metadata.json and at least one .keras model are present.
4. Run python app.py and open http://127.0.0.1:5000.
5. For hosted camera access, use HTTPS and verify the browser camera permission flow.
6. Before submission, record the tested environment, model filename, test images, observed outputs, and any deployment errors.
7. Complete the programme, branch, roll numbers, academic session, and submission date; confirm contribution roles, verify dataset provenance, and obtain institutional approval.

Appendix D Technical terminology

Term	Meaning in this project
Augmentation	Random image transformations applied during model training to vary examples.
Backbone	The pretrained MobileNetV2 feature-extraction network.
Class weight	A training-loss multiplier derived from class frequency.
Fine-tuning	Updating selected pretrained layers at a low learning rate for the target task.
Inference	Using the saved model to calculate scores for a new input image.
PWA	Progressive Web App metadata and shell behavior for an installable web experience.
Softmax score	A normalized model output across the configured classes; not automatically calibrated confidence.
Validation set	Data used during development to monitor and select a model; distinct from an independent test set.
WSGI	Python server interface used by Gunicorn to host the Flask application.

Appendix E Recommended independent evaluation protocol

1. Define the intended user, target device types, capture conditions, and inclusion/exclusion rules.
2. Set aside a test set before model selection; avoid near-duplicate leakage from training images.
3. Record consent/provenance and labels, then have ambiguous examples reviewed against a written class guide.
4. Report sample counts and class-wise precision, recall, F1, support, confusion matrix, and aggregate metrics.
5. Stratify results by lighting, background, viewpoint, object condition, device, and single-item versus mixed scenes.
6. Measure end-to-end request time and failure rate on the target hosting configuration separately from model accuracy.
7. Publish failure examples and known limitations; do not use the test set for repeated tuning.

Appendix F College submission checklist

- Confirm degree/programme, department, semester, academic session, roll numbers, and approved project title.
- Verify all student names and spellings on cover, declaration, certificate, presentation, and forms.
- Confirm the proposed contribution roles with each team member.
- Ask the mentor/HOD and co-mentor to review and sign institutional pages as required.
- Verify dataset source, license, and permission before redistribution.
- Replace preview-only screenshots with live app screenshots if the department requires runtime evidence.
- Run and document the recommended software, camera, mobile layout, and deployment tests.
- Attach the final repository commit or archive and make sure report metrics match the saved model metadata.

This report is a technical project document prepared from the supplied repository snapshot. Institutional format, declarations, signatures, and approval remain subject to the department's requirements.